A Project Report

On

Agentic AI Procurement Solution

Submitted in partial fulfilment of the requirements of

University of Mumbai

For the Degree of

Bachelor of Engineering

in

COMPUTER ENGINEERING

Submitted by

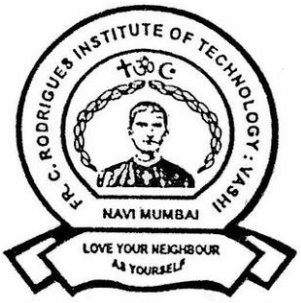
More Anjali Dinesh
Sankaye Vaidehi Jagdeep
Sawant Niraj Ravindra
Shete Anooj

Supervised by

Dr. Jyoti More

&

Mr. Alok
(External guide)



Department of Computer Engineering
Fr. Conceicao Rodrigues Institute of Technology
An Autonomous Institute & Permanently Affiliated to University of Mumbai Sector 9A, Vashi, Navi Mumbai - 400703

UNIVERSITY OF MUMBAI

APPROVAL SHEET

This is to certify that the project entitled

Agentic AI Procurement Solution

Submitted by

More Anjali Dinesh	**1023207**
Sankaye Vaidehi Jagdeep	**1023239**
Sawant Niraj Ravindra	**1023240**
Shete Anooj	**1023250**

Supervisor / Project Guide	**Project Coordinator**
Internal Examiner	**External Examiner**
Head of Department	**Principal**

COLLEGE SEAL

Date : _____

Place : _____

Declaration

We declare that this written submission for B.E. Declaration entitled "**Agentic AI Procurement Solution**" represents our ideas in our own words and where others' ideas or words have been included. We have adequately cited and referenced the sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any ideas/data/fact/source in our submission. We understand that any violation of the above will cause disciplinary action by the institute and also evoke penal action from the sources which have thus not been properly cited or from whom permission has not been taken when needed.

Project Group Members:

1. More Anjali Dinesh, 1023207

_____

2. Sankaye Vaidehi Jagdeep, 1023239

_____

3. Sawant Niraj Ravindra, 1023240

_____

4. Shete Anooj, 1023250

_____

Abstract

IT procurement covers the purchase, licensing, renewal and risk assessment of hardware, software and SaaS services. In many organisations it is still handled through emails, spreadsheets and separate tools, which leads to slow approvals, duplicate purchases, unused licences, missed contract renewals and weak visibility of vendor risk. Existing automation techniques, such as OCR and rule-based extraction, approval workflows and machine learning, usually address only one stage of the procurement cycle, so there is a need for a system that coordinates several stages with limited human intervention. Agentic AI suits this setting because specialised agents can each own one part of the process and cooperate through events.

This report presents an Agentic AI-Based IT Procurement Intelligence Platform built as a modular, event-driven system of four cooperating agents: Document and Vendor Intelligence, Approval and Inventory Intelligence, Contract and Risk Analysis, and a Platform agent made up of the notification and authentication services. The services are implemented in FastAPI and communicate over a Kafka-compatible event bus (Redpanda), with PostgreSQL, Redis and MinIO as the shared data layer, Temporal for durable workflows, MLflow for model tracking, Prometheus and Grafana for monitoring, and a React and TypeScript frontend served through an Nginx gateway.

Uploaded documents are scanned for malware, parsed with Docling (with pdfplumber and Tesseract OCR as fallbacks), classified as purchase orders, invoices or quotations, and processed by regular-expression field extraction. Vendors are matched with fuzzy name matching, duplicate invoices are detected using amount and date tolerances, and low-confidence results are sent to a human review queue. Purchase requests follow spend-tier approval chains with SLA-based escalation, and inventory is reserved with Redis locks. Licence utilisation drives automatic reclaim requests. Contracts are generated from templates, key clauses are extracted by rules, signature callbacks are verified with HMAC and replay protection, and vendor risk is scored by a Random Forest classifier whose drift is monitored using the Population Stability Index. The implementation is covered by 134 automated unit tests and a 12-step end-to-end test. The platform is intended to reduce manual processing, avoid unnecessary spending, and give procurement teams explainable insight into vendor and procurement risk.

Contents

Abstract iii

List of Figures vi

List of Tables vii

1 Introduction 1
- 1.1 Background . . . 2
- 1.2 Motivation . . . 2
- 1.3 Aim and Objectives . . . 2
  - 1.3.1 Aim . . . 2
  - 1.3.2 Objectives . . . 2
- 1.4 Problem statement . . . 3
- 1.5 Scope . . . 3
- 1.6 Report Outline . . . 4

2 Study of the System 5
- 2.1 About the Technique . . . 6
- 2.2 Various Available Techniques . . . 6
  - 2.2.1 Document Intelligence . . . 6
  - 2.2.2 Document Classification . . . 7
  - 2.2.3 Vendor Deduplication and Identity Verification . . . 7
  - 2.2.4 Workflow Orchestration and Concurrency Control . . . 8
  - 2.2.5 Usage Anomaly Detection and Explainability . . . 8
  - 2.2.6 Contract Analysis and Clause Extraction . . . 8
  - 2.2.7 Vendor Risk Scoring and Drift Monitoring . . . 8
  - 2.2.8 Multi-Agent and Agentic Architectures . . . 9
- 2.3 Related Works . . . 9
- 2.4 Literature Summary and Research Gap . . . 9

3 Proposed System 12
- 3.1 Problem Statement . . . 13
- 3.2 Scope . . . 13
- 3.3 Scope Design . . . 14
- 3.4 Proposed System . . . 15
  - 3.4.1 Overview and Architecture . . . 15
  - 3.4.2 The Four Agents . . . 15
  - 3.4.3 Document Processing Pipeline . . . 16
  - 3.4.4 Inter-Agent Communication and Workflow . . . 17
  - 3.4.5 Use Case View . . . 19

3.4.6 Technology Stack . . . 19
3.4.7 Key Design Principles . . . 21

4 Design of the System 22
4.1 Requirement Engineering . . . 23
4.1.1 Requirement Elicitation . . . 23
4.1.2 Software Lifecycle Model . . . 23
4.1.3 Requirement Analysis . . . 24
4.1.3.1 Functional Requirements . . . 24
4.1.3.2 Non-Functional Requirements . . . 26
4.1.4 UML Diagrams and Data Flow Diagrams . . . 27
4.1.4.1 Use Case Diagram . . . 27
4.1.4.2 Data Flow Diagram – Level 0 . . . 27
4.1.4.3 Data Flow Diagram – Level 1 . . . 27
4.1.4.4 Sequence Diagram . . . 27
4.1.4.5 Activity Diagram: Document Processing . . . 29
4.1.4.6 Class/Entity Relationship Diagram . . . 29
4.1.5 Cost Analysis . . . 29
4.1.6 Hardware and Software Requirements . . . 31
4.2 System Architecture . . . 32
4.2.1 UI/UX Design . . . 32
4.2.2 Block Diagram . . . 33
4.2.3 Main Data Paths . . . 34

5 Result and Discussion 37
5.1 Screenshots of the System . . . 38
5.2 Sample Code (of imp part/ main logic) . . . 38
5.3 Testing . . . 38
5.3.1 Unit Testing . . . 38
5.3.2 Integration Testing . . . 38
5.3.3 Blackbox Testing . . . 39

6 Conclusion & Future Scope 40

References 42

Acknowledgement 42

Appendix A: Timeline Chart 44

Appendix B: Sustainable Development Goal (SDG) Mapping 46

Appendix C: Complex Engineering Problem Identification and Evaluation 47

Appendix D: Project Outcomes 50

List of Figures

3.1 Overall architecture of the proposed system 15
3.2 Document processing pipeline of the Document and Vendor Intelligence agent 17
3.3 Event flow between the agents 18
3.4 Use case diagram of the IT procurement system 19

4.1 Iterative and incremental development cycle 24
4.2 Level 0 data flow diagram (context diagram) 27
4.3 Level 1 data flow diagram. Processes 2.0–5.0 exchange events through the messaging layer, and process 6.0 consumes them. 28
4.4 Sequence diagram of the purchase-request journey 29
4.5 Activity diagram of document processing 30
4.6 Entity relationship diagram (main entities of the shared schema) 30
4.7 Main-screen wireframes of the platform 34
4.8 Block diagram of the proposed system architecture 34

List of Tables

2.1 Traditional procurement practices and their shortcomings 7
2.2 Comparison of related works 10
2.3 Comparison of related works (continued) 10
2.4 Techniques reviewed and their use in the proposed system 11

3.1 Problems in the current procurement process and their impact 13
3.2 Summary of the four agents in the proposed system 16
3.3 Main events exchanged between agents 18
3.4 Technology stack of the proposed system 20

4.1 Functional requirements 24
4.2 Non-functional requirements 26
4.3 Level 1 DFD processes 28
4.4 Indicative cost analysis 31
4.5 Hardware requirements 31
4.6 Software requirements 32
4.7 Principal screens and their purposes 33
4.8 Architectural layers 35

5.1 Unit tests per service 38
5.2 Black-box test cases 39

Chapter 1

Introduction

1.1 Background

IT procurement is an important part of an organisation's operations as it involves the acquisition and management of hardware, software licences, SaaS subscriptions and related services. The procurement process involves several stages, including vendor identification, document processing, approval, inventory management, contract handling and vendor risk assessment. In many organisations these activities are handled through emails, spreadsheets, invoices and separate systems, making it challenging to maintain a consistent and updated view of procurement activities. As a result, approvals can be delayed, purchases duplicated and licences left unused, contracts are difficult to track, and vendor-related risks increase. Artificial intelligence and automation offer an opportunity to improve these activities and make the procurement process more efficient.

1.2 Motivation

The motivation for this project is the need to reduce the time spent on manual IT procurement tasks and to increase transparency and control during the process. Employees currently have to verify documents, review vendor details, request and obtain approvals, check inventory and licence utilisation, and keep track of contract renewals. These tasks become difficult when the volume of data is high. A system that automates and optimises them, and supports communication between the different functions, would be of great benefit. The project therefore implements a set of special-purpose software agents that assist with document processing, vendor matching, approvals, inventory and licence analysis, contract management and risk detection.

1.3 Aim and Objectives

1.3.1 Aim

To develop an Agentic AI-Based IT Procurement Intelligence Platform that integrates, automates and intelligently manages the major activities of the IT procurement lifecycle while maintaining human oversight for critical decisions.

1.3.2 Objectives

- To process procurement documents such as purchase orders, invoices and quotations and extract their relevant information with a confidence score for each result.
- To match vendors against existing records and to identify duplicate vendors and duplicate invoices.
- To automate approval workflows based on spend tiers, SLA deadlines and inventory availability, with escalation when an approver does not respond.
- To identify underutilised software licences and create reclaim requests for them.
- To assist in contract management by generating contracts, extracting renewal-related clauses and issuing timely renewal reminders.
- To assess vendor risk continuously using a machine learning model and to monitor that model for drift.

- To provide shared platform services such as authentication, role-based access control, notifications, monitoring and audit logging.
- To keep humans involved in low-confidence results and in critical decisions such as changes to vendor payment details.

1.4 Problem statement

IT procurement involves several activities such as collecting procurement documents, verifying vendor information, processing invoices and quotations, obtaining approvals, managing software licences, monitoring contracts and assessing vendor risks. These activities are often handled through multiple systems and manual processes, resulting in delays, duplicate records, inefficient resource utilisation and limited visibility across the procurement lifecycle.

Traditional procurement systems primarily record and manage transactions rather than analysing procurement data and assisting users in decision-making. Identifying duplicate vendors or invoices, detecting underutilised licences, tracking contract renewals and assessing vendor risk may require significant manual effort. Furthermore, automated decisions without a confidence assessment and human oversight can lead to incorrect procurement actions.

There is therefore a need for an integrated solution that can automate routine procurement activities, analyse relevant data, support approval decisions and monitor procurement risk. The proposed Agentic AI-Based IT Procurement Intelligence Platform addresses these challenges by combining document processing, vendor intelligence, workflow automation, licence management, contract monitoring, risk scoring and human-in-the-loop controls in one platform.

1.5 Scope

The scope of the platform covers the major activities of the IT procurement lifecycle:

- **Procurement Document Processing:** Scanning, parsing, classifying and extracting structured information from purchase orders, invoices and quotations.
- **Vendor Intelligence:** Matching extracted vendor names against existing records, creating new vendor records, and holding changes of bank or payment details for verification by a second user.
- **Invoice and Duplicate Detection:** Flagging invoices that are likely to be duplicates using vendor, amount tolerance, date window and document number checks.
- **Approval Workflow Automation:** Routing purchase requests through spend-tier approval chains with SLA timers, escalation and inventory reservation.
- **Software Licence Management:** Calculating licence utilisation and creating reclaim requests for underused licences.
- **Contract Management:** Generating contracts from templates, extracting renewal type, notice period and end date, handling e-signature callbacks, and scheduling renewal reminders.
- **Vendor Risk Assessment:** Scoring vendors with a machine learning model, listing the contributing factors, and monitoring the model for drift.

- **Shared Platform Services:** Authentication and role-based access control, email notifications, audit logging, metrics and monitoring.
- **Human-in-the-Loop Decision Making:** Routing low-confidence documents and payment-detail changes to authorised users.
- **Agentic AI Integration:** Using specialised agents that cooperate through events to coordinate tasks across the lifecycle.

The platform is intended to assist procurement teams by reducing manual effort, improving data accuracy, increasing process visibility and supporting informed decisions. It focuses on decision support and workflow automation rather than replacing human decision-makers in critical procurement activities.

1.6 Report Outline

This report is organised into six chapters. Chapter 1, Introduction, presents the background and motivation of the project, followed by the aim and objectives, problem statement, scope and report outline. Chapter 2, Study of the System, discusses the techniques relevant to the proposed system, the various available techniques, and related works in the field of IT procurement and Agentic AI. Chapter 3, Proposed System, describes the problem addressed by the project, its scope, and the proposed Agentic AI-Based IT Procurement Intelligence Platform. Chapter 4, Design of the System, covers requirement engineering, requirement elicitation, the software lifecycle model, requirement analysis, UML diagrams and DFDs, cost analysis, hardware and software requirements, system architecture, UI/UX design and the block diagram of the system. Chapter 5, Result and Discussion, presents the implemented system through screenshots, important code and main logic, and discusses the testing performed, including unit testing, integration testing and black-box testing. Chapter 6, Conclusion and Future Scope, summarises the project outcomes, limitations and possible future enhancements. The report also includes references and appendices covering the project timeline, SDG mapping, complex engineering problem identification and evaluation, and project outcomes.

Chapter 2

Study of the System

Procurement departments handle a steady stream of documents, including purchase orders, invoices and vendor quotations, and these rarely arrive in the same form twice. Some come by email, some through vendor portals and others are scans of printed pages. In many organisations someone still reads each one and types the details into a system by hand. Contracts are followed up in spreadsheets, vendors are vetted once at onboarding, and software licences are checked with a simple inactivity rule. Over time this leads to typing mistakes, duplicate payments, vendors whose identity was never properly confirmed, contracts that renew without anyone noticing, and licences that are paid for but never used. Fraud is a further concern. In Business Email Compromise (BEC), an attacker poses as a supplier and asks for bank details to be changed, and the request goes through because nobody verifies it independently.

This work studies a platform that tackles these problems through four agents communicating over events: Document and Vendor Intelligence, Approval and Inventory Intelligence, Contract and Risk Analysis, and a Platform agent for notifications, authentication and monitoring. Because each agent relies on a different set of techniques, the chapter covers a range of methods instead of a single one. It begins with how procurement is traditionally done and where that approach falls short, and then reviews the main techniques for each part of the problem, noting the strengths and weaknesses of each and which one the project adopted. Ten related research works are then compared, and the chapter ends with a summary of the literature and the gap this project sets out to address.

2.1 About the Technique

Traditional procurement processing rests on four habits. The first is manual or template-based data entry: staff key in invoice and purchase order details, or an OCR template is built for each vendor's layout. The second is static approval rules, where a request follows a fixed chain with no deadline, no escalation and no awareness of the remaining stock. The third is periodic vendor review, which in practice often means one questionnaire at onboarding. The fourth is the flat-threshold licence audit, in which a seat is flagged if nobody has logged in for some fixed number of days.

These habits work well enough when volumes are small, but they break down quickly as an organisation grows. Table 2.1 summarises where each of them struggles.

These gaps motivate an automated, event-driven approach. The idea is not to hand everything to a model. Pretrained or off-the-shelf components handle the perception-heavy work such as reading document layouts, a small amount of custom training is used where a task is specific to this project (the vendor risk model), and plain rules are kept where certainty and explainability matter, such as spend-tier routing, document classification and clause extraction.

2.2 Various Available Techniques

2.2.1 Document Intelligence

Template-based and regular-expression approaches are quick and predictable, but they are brittle when vendors change their layouts. OCR engines such as Tesseract [13] and PaddleOCR [4] can extract text from scanned documents, but they lose table structure in the process. Layout-aware parsers such as Docling [3] recover the reading order and table structure of a document, although they are not optimised for invoices. Multimodal

Table 2.1: Traditional procurement practices and their shortcomings

Area	Traditional approach	Shortcoming
Data capture	Manual entry and per-vendor templates	Slow and error-prone; needs updating whenever a vendor changes its document layout.
Vendor verification	Manual lookup and email confirmation	Vulnerable to Business Email Compromise (BEC) fraud and duplicate vendor records.
Approvals	Email chains and fixed rules	No SLA timers or escalation; conflicts in stock allocation when requests overlap.
Licence management	Fixed inactivity threshold	Ignores usage patterns and may wrongly flag seasonal drops in activity.
Contract tracking	Calendar reminders	Notice periods may be missed, resulting in unintended automatic renewals.
Risk assessment	One-time questionnaire	Becomes outdated and may miss new adverse information about a vendor.

transformers, particularly those in the LayoutLM family [1], [2], achieve the best results on forms and invoices, but they need labelled data to fine-tune and more compute to run. This project uses Docling as the primary parser for every PDF (it detects scanned pages and applies its bundled OCR), with pdfplumber as a lightweight fallback if Docling fails, and Tesseract (through pytesseract) for standalone scanned images. PaddleOCR was evaluated for the image path but rejected, because its native inference engine aborted the host process on every version pairing tried. The LayoutLM models were studied but not implemented.

2.2.2 Document Classification

TF-IDF with logistic regression is light, fast to train and easy to inspect, which makes it a candidate for a three-class problem (purchase order, invoice, quotation). Transformer classifiers such as BERT have a higher ceiling but need more data and compute than this task justifies. Because the three document types carry strong, distinctive phrases ("tax invoice", "purchase order", "valid until"), the project uses a weighted keyword classifier: title-like phrases score higher than incidental words, and the margin between the winning and runner-up class is converted into a confidence value. It is deterministic, easy to explain and easy to extend, but it cannot learn from data and depends on the vocabulary it has been given.

2.2.3 Vendor Deduplication and Identity Verification

Fuzzy name matching (rapidfuzz) tolerates spelling differences and corporate suffixes but can produce false matches when used on its own. Official identifiers such as GSTIN and IFSC are more reliable, but live lookups are limited by API quotas. The project therefore normalises vendor names, matches them with a fuzzy ratio threshold, and adds

two governance controls around the vendor record: malware scanning of every upload with ClamAV (signature-based, so quick but with no protection against zero-day threats), and dual control of any change to bank or payment details. Registry-based identifier verification is left as future scope.

2.2.4 Workflow Orchestration and Concurrency Control

Cron jobs and scripts are simple to set up, but they are not durable and do not handle deadlines well. Temporal offers durable timers, signals, escalation and replay, but has a steeper learning curve. Redis locks and database-level locks stop two requests from claiming the same stock at once, but need careful handling of failures. Kafka-compatible brokers provide decoupled, replayable events between agents, provided the consumers are idempotent [11]. The project uses Temporal for approval, renewal and drift-check workflows, Redis locks for inventory reservation, and Redpanda as the event bus.

2.2.5 Usage Anomaly Detection and Explainability

A flat inactivity threshold is transparent but flags seasonal and weekend dips by mistake. Isolation Forest [8] is unsupervised and efficient on tabular data, though it is sensitive to its contamination setting and has no built-in explanations, and SHAP [9] supplies per-feature contributions at extra computational cost. The project measures licence utilisation as the share of seats with at least one login in a 30-day window (also tracking 60 and 90 days), and compares it with configurable warning and reclaim thresholds. This is easy to explain to a licence owner; anomaly-based scoring with Isolation Forest and SHAP is a possible extension.

2.2.6 Contract Analysis and Clause Extraction

Keyword search misses paraphrased clauses. Legal-BERT [6] fine-tuned on CUAD [5] handles legal wording better and answers one question per field, but it was trained mostly on US-style contracts and can only extract text that exists. FinBERT [7] scores financial sentiment in adverse media, although sentiment is only a rough proxy for risk. Because the clauses needed here (renewal type, notice period, end date) follow a small number of standard phrasings, and because the contracts are generated from the platform's own templates, the project uses rule-based extraction with regular expressions and date parsing. The rules are deterministic and testable; validating them against CUAD is future scope.

2.2.7 Vendor Risk Scoring and Drift Monitoring

Static questionnaires are only snapshots, whereas a learned classifier can combine several signals into a score. Random Forests [12] are robust on small tabular datasets, handle mixed feature types and provide feature importances that can be used for explanation. The project trains a Random Forest on six vendor features and maps the predicted probability to Low, Medium and High bands. The Population Stability Index (PSI) compares the distribution of recent scores with the training-time distribution and warns of model drift, but it does not retrain the model, so a human decides what to do with the signal.

2.2.8 Multi-Agent and Agentic Architectures

Splitting the workflow among specialised agents gives modularity and clear ownership, at the cost of integration effort. This platform manages that through a shared event contract, a common REST response envelope, shared authentication middleware, idempotent database migrations and a single gateway.

2.3 Related Works

To see where this system fits, ten published works were examined, each covering one building block of the platform. They are grouped below by the problem they address, and Tables 2.2 and 2.3 then compare them.

Document understanding. LayoutLM [1] was among the first models to learn from text and its two-dimensional position on the page together, and it improved results on forms, receipts and invoices considerably. Its successor, LayoutLMv3 [2], added image information through a shared text-and-image masking objective and dropped the CNN backbone, giving a simpler and stronger model. On the tooling side, Docling [3] offers an open-source pipeline that rebuilds reading order and table structure locally, and PaddleOCR 3.0 [4] provides accurate multilingual OCR with structure analysis, although it runs best on a GPU.

Contract and risk language. CUAD [5] gives 510 expert-annotated contracts covering 41 clause types and has become the standard benchmark for clause extraction. LEGAL-BERT [6] showed that pretraining on legal text clearly outperforms generic BERT on legal tasks. For the financial side of vendor risk, FinBERT [7] contributes sentiment analysis tuned to financial language.

Anomaly detection and explanation. Isolation Forest [8] finds unusual records by how quickly random splits separate them from the rest, so it needs no labelled examples. SHAP [9] then explains a single prediction by showing how much each feature pushed the score up or down.

Agent frameworks. AutoGen [10] stands for the popular LLM-driven approach in which agents cooperate by conversation. It is included mainly as a contrast to the design chosen here.

Taken together, the works show a clear pattern. The document-AI papers are mainly about how accurately fields can be read, and they say little about what should happen next, such as matching the vendor, catching duplicates or holding suspicious changes for review. This platform builds a confidence-gated pipeline around the extraction step. The legal NLP papers stop at pulling clauses out of text, while here the extracted terms feed a renewal workflow that sends reminders and sits in the same system as a live vendor risk score. Isolation Forest and SHAP are general-purpose tools, so a simpler, explainable utilisation rule was preferred for licence reclaim. Finally, agent frameworks like AutoGen prioritise flexibility and free-form dialogue, whereas procurement needs predictable, auditable and concurrency-safe behaviour. That is why the project relies on a Kafka-compatible event bus [11] for events and Temporal workflows for orchestration.

2.4 Literature Summary and Research Gap

Table 2.4 pulls together the techniques reviewed and shows where each one stands in the proposed system.

Table 2.2: Comparison of related works

Reference	Contribution	Advantages	Limitations	Role in this project
Xu et al., LayoutLM (KDD 2020) [1]	Joint pretraining of text and 2D layout for document images	Large gains on forms, receipts and invoices	No image features in pretraining; needs fine-tuning	Studied; motivates layout-aware parsing
Huang et al., LayoutLMv3 (ACM MM 2022) [2]	Unified text-image masking, no CNN backbone	State-of-the-art results; simpler architecture	Compute-heavy; needs labels for new document types	Studied; not implemented (future scope)
Auer et al., Docling (2024) [3]	Open-source layout, reading-order and table recovery	Fast, local, strong on tables	Not tuned to invoices; weak on poor scans	Primary PDF parser (implemented)
Cui et al., PaddleOCR 3.0 (2025) [4]	OCR and structure-analysis toolkit	Accurate, multilingual	GPU preferred; crashed in the test environment	Evaluated and rejected; Tesseract used
Hendrycks et al., CUAD (NeurIPS 2021) [5]	510 contracts, 41 clause categories	Realistic benchmark, CC BY 4.0	English, mostly US contracts	Reference for clause types; validation is future scope

Table 2.3: Comparison of related works (continued)

Reference	Contribution	Advantages	Limitations	Role in this project
Chalkidis et al., LEGAL-BERT (EMNLP 2020) [6]	BERT pretrained on legal corpora	Beats generic BERT on legal tasks	Smaller corpus; mostly English	Studied; rule-based extractor used instead
Araci, FinBERT (2019) [7]	BERT further pretrained on financial text	Strong financial sentiment	Sentiment is only a proxy for risk	Studied; possible adverse-media signal later
Liu et al., Isolation Forest (ICDM 2008) [8]	Anomaly detection by random partitioning	Linear time, no labels, high-dimensional data	No native explanation; contamination must be set	Studied; utilisation thresholds used
Lundberg and Lee, SHAP (NeurIPS 2017) [9]	Unified feature attribution	Consistent local explanations; fast for trees	Cost grows with features; not causal	Studied; importance-weighted factors used
Wu et al., AutoGen (2023) [10]	Multi-agent LLM conversation framework	Modular and customisable	Conversation-driven; hard to audit	Contrast: event-driven agents with durable workflows

Table 2.4: Techniques reviewed and their use in the proposed system

Area	**Best-known technique**	**Use in this system**
Layout and OCR	Docling, PaddleOCR, Tesseract	Docling for PDFs, pdfplumber fallback, Tesseract for images
Extraction cross-check	LayoutLMv3	Not implemented; future scope
Classification	TF-IDF + logistic regression	Weighted keyword classifier with confidence margin
Clause extraction	Legal-BERT on CUAD	Rule-based extraction of renewal type, notice period and end date
Licence analysis	Isolation Forest + SHAP	Utilisation ratio with warning and reclaim thresholds
Risk scoring	Classifier + FinBERT	Random Forest on six features, Low/Medium/High bands, PSI drift check
Orchestration	Temporal, Redis locks, Kafka	Temporal workflows, Redis reservation locks, Redpanda event bus

The main thing the literature lacks is integration. Document extraction, clause analysis, anomaly detection and workflow orchestration are usually studied as separate problems and judged on benchmarks. Very few systems join them into a single governed procurement pipeline, and fewer still do so with three properties in mind. The pipeline should be *fraud-aware*, with dual-control approval of payment-detail changes and malware scanning that fails closed. It should be *accountable*, with an audit log of document, vendor, approval and contract actions, and monitoring for model drift. And it should be *observable*, with per-service metrics, dashboards and a notification log so that the state of every request can be inspected. This project is an attempt to fill that gap.

Chapter 3

Proposed System

This chapter states the problem that motivated the project, defines what the work will and will not cover, and describes the proposed system in terms of its architecture, agents, document pipeline, event flow and technology choices.

3.1 Problem Statement

In most organisations the IT procurement process is a chain of loosely connected manual steps. A purchase order or invoice arrives in whatever form the vendor chose, someone retypes its contents into a system, someone else checks the vendor, and an approval request travels by email until somebody replies. Contracts are filed with their renewal dates noted on a calendar, and software licences are renewed without checking whether anyone still uses them. Each step is slow on its own, and together they leave room for costly mistakes and for fraud. Table 3.1 lists the main problems and what they cost.

Table 3.1: Problems in the current procurement process and their impact

Problem	**Impact**
Documents arrive through disconnected channels in inconsistent formats	Manual data entry, delays and extraction errors
Vendor records are not de-duplicated and payment-detail changes are accepted on request	Duplicate vendors, Business Email Compromise (BEC) fraud and payments to the wrong account
Approvals are routed by hand with no SLA or stock checks	Stalled requests, overspending and two requests consuming the same stock
Software licence usage is not monitored, or is judged by a flat inactivity rule	Money spent on unused seats
Contract terms are tracked informally	Silent auto-renewals and missed notice periods
Vendor risk is assessed once, at onboarding	Stale risk ratings
No single view of a request across tools	Difficult auditing and slow investigation of problems

These problems are connected. A vendor record that was never properly checked is exactly what makes a fraudulent bank-detail change possible, and an extraction error that goes unnoticed can end up as a duplicate payment. Fixing one step in isolation therefore helps only a little.

The problem addressed in this project can be stated as follows: to design and build an automated, event-driven procurement platform that reads unstructured procurement documents, matches every document to a vendor record, routes approvals under spend and deadline controls, manages contracts and renewals, and keeps a vendor risk score up to date, while keeping automated decisions explainable and auditable and keeping a human in the loop for low-confidence results and sensitive changes.

3.2 Scope

The project covers the path from an incoming procurement document to a signed contract. Within this scope the system scans and parses documents, matches vendors, routes

approvals with spend-tier and SLA controls, reserves inventory, tracks licence utilisation, generates and tracks contracts, scores vendor risk, and keeps users informed through email notifications, all behind role-based access and with metrics-based monitoring.

The work does not include payment execution, ERP integration, registry-based verification of tax and bank identifiers (GSTIN and IFSC), sanctions screening, a live e-signature provider, or automatic retraining of the models. The vendor risk model is trained on a documented synthetic dataset, the licence usage data is generated synthetically, and the sample documents are synthetic.

3.3 Scope Design

The system design defines the structure, components, interfaces and information flow of the proposed platform. It explains how the major modules interact to support document processing, vendor assessment, approval, contract handling and decision support, and it gives a structured view of how the components exchange information.

The scope of the system design includes the following areas:

- **System architecture:** Defines the major components of the platform and the relationships between the user interface, gateway, backend services, data stores and the event bus.
- **User interface design:** Describes the role-specific screens and interaction flow through which users submit requests, review extracted data, approve or reject requests and inspect vendors, contracts and notifications.
- **Agent coordination:** Describes how the specialised agents cooperate through events, for example document intake, approval, contract generation and notification.
- **Data flow:** Represents how information moves from user input through processing and analysis to the final output.
- **External dependencies:** Identifies the external components the platform relies on, namely the ClamAV virus-definition updates, an SMTP service for email (Mailpit in development) and an e-signature provider that calls back through a signed webhook.
- **Output and decision support:** Describes how the platform presents confidence scores, risk bands with contributing factors, and utilisation figures to help users make informed decisions.

The system is intended to assist procurement decisions; it does not guarantee the accuracy of extraction in every case, which is why low-confidence results are sent for human review. Automated payment execution, complete ERP integration, a production e-signature service and automatic model retraining are outside the scope.

This scope establishes the boundaries of the design and provides a basis for describing the system architecture, interface design, component diagrams and data-flow diagrams in the relevant sections of the report.

3.4 Proposed System

3.4.1 Overview and Architecture

The proposed system is an event-driven platform made up of four cooperating agents. Each agent is an independent FastAPI service with its own responsibilities, and the agents communicate mainly through events on a Kafka-compatible bus (Redpanda) instead of calling each other directly. Users reach the system through a React frontend and an Nginx API gateway, and every service validates requests with a shared JWT authentication and role-based access control middleware. All services use a common PostgreSQL database, Redis for locks, and MinIO for document storage. Figure 3.1 shows the overall structure.

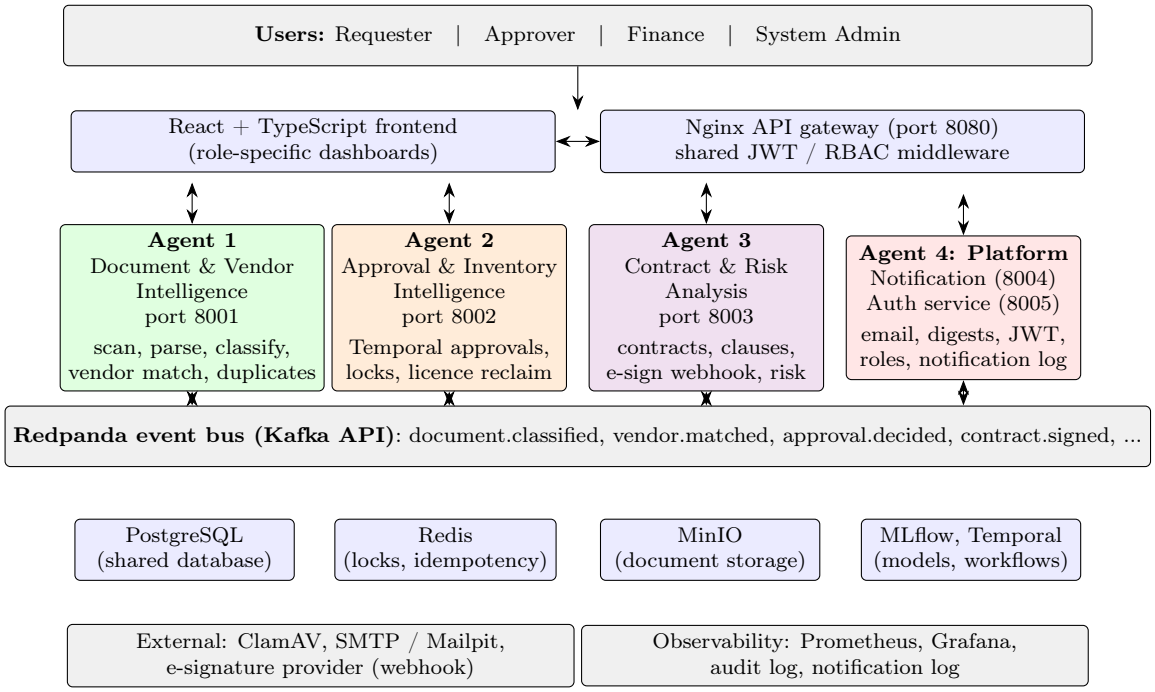


Figure 3.1: Overall architecture of the proposed system

Splitting the system this way keeps each concern separate. A change to the extraction logic does not touch the approval engine, and a slow document parse cannot block a user's upload, because the heavy work runs asynchronously in a background worker off the request path.

3.4.2 The Four Agents

Table 3.2 summarises the agents, and the paragraphs that follow describe what each one does.

Document and Vendor Intelligence. This agent is the entry point for every document. It streams each upload through ClamAV and rejects it if the scanner flags the file or cannot be reached, then stores accepted files in MinIO and publishes a `document.ingested` event. A background worker extracts structured data and gives each field a confidence score so that doubtful results go to a human reviewer instead of being accepted automatically. The agent also matches the vendor against existing records, detects duplicate invoices, and never applies a change of bank or payment details directly: such a change waits in a pending state until a different user with the finance or admin role confirms it.

Table 3.2: Summary of the four agents in the proposed system

Agent	Port	Main responsibility	Key techniques
Document and Vendor Intelligence	8001	Document intake, extraction, vendor matching, duplicate detection, payment-change control	Docling, pdfplumber, Tesseract, keyword classifier, regex extraction, rapidfuzz, ClamAV
Approval and Inventory Intelligence	8002	Approval routing, SLA escalation, inventory reservation, licence reclaim	Temporal, Redis locks, utilisation scoring
Contract and Risk Analysis	8003	Contract generation, e-signature callback, clause extraction, renewal reminders, vendor risk scoring, offboarding	Jinja2, rule-based clause extraction, scikit-learn Random Forest, MLflow, PSI drift check
Platform (Notifications and Auth)	8004 / 8005	Authentication and RBAC, notifications, notification log	JWT, bcrypt, Kafka consumers, Jinja2, SMTP, Prometheus

Approval and Inventory Intelligence. This agent derives the approval chain for each request from configurable spend tiers, and it starts a Temporal workflow that waits for an approve or reject signal up to an SLA timeout (48 hours by default) and escalates automatically if the approver does not respond. It reserves stock with Redis locks so that two concurrent requests cannot take the last unit, and it splits an oversubscribed request into an immediate part and a back-ordered part. A utilisation job calculates, for every active licence, the share of seats used in the last 30 days; when utilisation falls below the reclaim threshold (30% by default) it creates a reclaim request for approval and publishes a `license.usage.updated` event. When it receives `contract.signed` it marks the originating request as fulfilled.

Contract and Risk Analysis. Once a request is approved, this agent fills a Jinja2 contract template (hardware purchase, SaaS subscription or professional services) and sends it for signature. Rule-based extraction reads the contract text to pull out the renewal type, notice period and end date, and a Temporal workflow raises renewal reminders at 60, 30 and 15 days before the notice deadline. The signature callback is accepted only if its HMAC signature is valid and the event has not been processed before. The same agent scores every vendor with a Random Forest model, lists the top contributing factors, publishes `risk.score.updated`, runs a weekly PSI drift check, and handles vendor offboarding.

Platform. The fourth agent handles what no single functional agent should own. The authentication service issues JWT access and refresh tokens for the four roles, and the notification service consumes the events of the other agents, renders templated emails, sends urgent messages immediately and batches routine ones into digests, and keeps a searchable log of every message sent, queued or failed. Both services expose Prometheus metrics.

3.4.3 Document Processing Pipeline

The most involved part of the system is the pipeline that turns an uploaded file into a trustworthy structured record, shown in Figure 3.2. The upload returns promptly, and a

Kafka consumer runs the stages in the background. Internally the stages are implemented as a chain of small agents that pass a JSON envelope from one to the next.

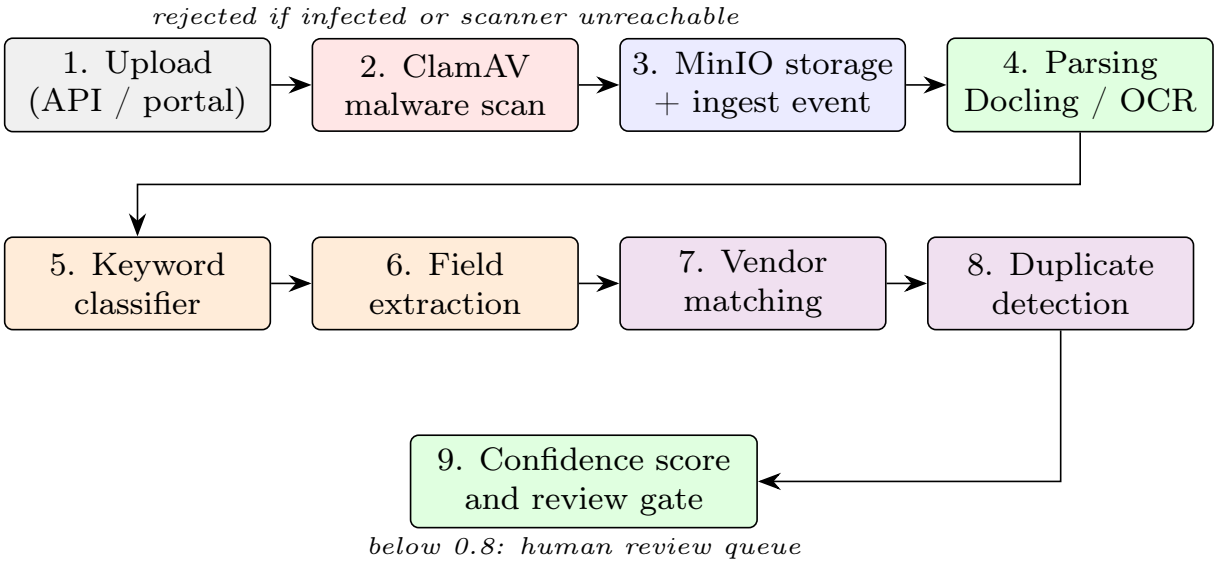


Figure 3.2: Document processing pipeline of the Document and Vendor Intelligence agent

A file is first streamed through ClamAV and rejected if it is flagged or if the scanner cannot be reached, so the system fails closed. After it is stored, the worker parses it: Docling recovers layout and tables for every PDF and applies its bundled OCR to scanned pages, with pdfplumber as a fallback, and Tesseract reads standalone scanned images. A weighted keyword classifier labels the document as a purchase order, invoice or quotation, and regular expressions extract the vendor name, line items, totals, dates and document number. The vendor is then normalised and matched against existing records with a fuzzy ratio (a threshold of 88 by default), and a new vendor record is created if no match is found. For invoices, the duplicate check compares vendor, amount (within 1%), date (within five days) and document number similarity. Finally, each field receives a confidence score that is discounted by the quality of the extracted text, and a document whose overall confidence is below 0.8 is placed in the review queue instead of being completed automatically. The worker publishes `document.classified` and `vendor.matched`.

3.4.4 Inter-Agent Communication and Workflow

Figure 3.3 shows how the agents exchange events, and Table 3.3 lists the main ones. Every message uses the same envelope (event ID, event type, timestamp, source service and payload), and the topics and payload fields are fixed in a shared contract document so that producers and consumers can be built in parallel.

In a typical run, a requester uploads a document and Agent 1 publishes `document.classified` together with `vendor.matched`. Agent 2 builds the approval chain for the purchase request and, once the final approver agrees, publishes `approval.decided`. Agent 3 then generates the contract and sends it for signature, and its `contract.signed` event tells Agent 2 to mark the request as fulfilled. The notification agent listens to these events and sends urgent alerts, such as an approval request or a flagged payment-detail change, immediately while batching routine ones into a digest.

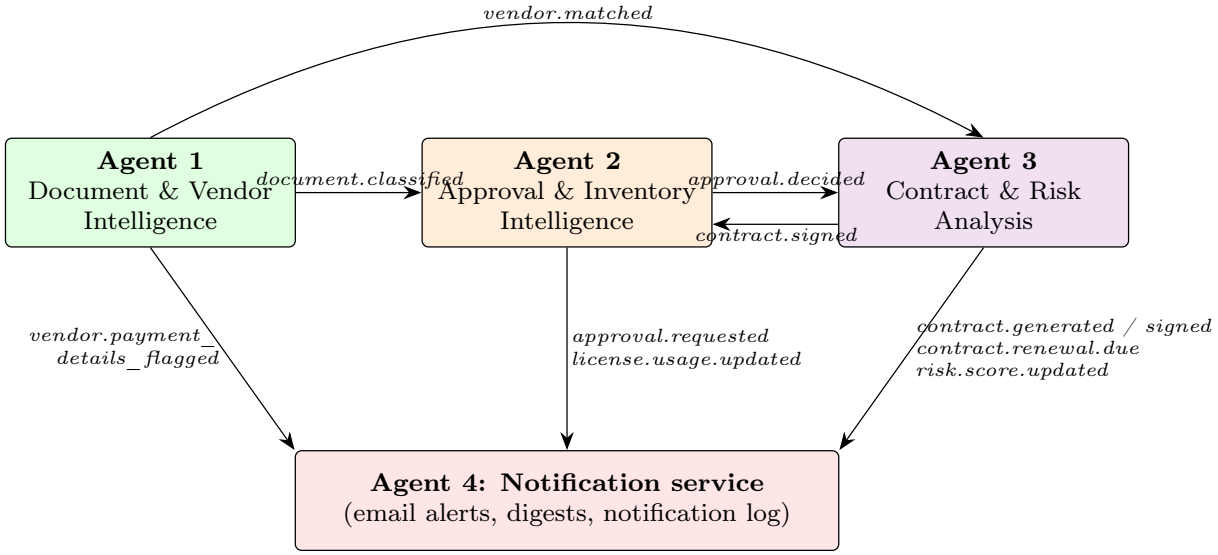


Figure 3.3: Event flow between the agents

Table 3.3: Main events exchanged between agents

Event	**Published by**	**Consumed by and purpose**
document.ingested	Agent 1	Agent 1 worker: starts the extraction pipeline
document.classified	Agent 1	Agent 2 and notifications: structured extraction result
vendor.matched	Agent 1	Agent 3: matched vendor identity for risk scoring
vendor.payment_ details_flagged	Agent 1	Notifications: alert on a pending payment-detail change
approval.requested / approval.decided	Agent 2	Agent 3 and notifications: drives contract generation and alerts
license.usage.updated	Agent 2	Notifications: utilisation alerts
contract.generated / contract.signed	Agent 3	Agent 2 and notifications: fulfilment closure
contract.renewal.due	Agent 3	Notifications: 60, 30 and 15 day renewal reminders
risk.score.updated	Agent 3	Notifications: risk alerts (urgent for the High band)
vendor.offboarded	Agent 3	Notifications: offboarding summary and contracts flagged
notification.send	Any service	Notifications: generic fallback message

3.4.5 Use Case View

Figure 3.4 presents the use case diagram of the system. Four roles take part: the Requester, who submits purchase requests and uploads documents; the Approver, who approves or rejects requests; the Finance user, who approves higher-value requests, verifies payment-detail changes and recomputes vendor risk; and the System Administrator, who generates and signs contracts, offboards vendors and reviews audit activity and system health. The use cases are grouped into five phases.

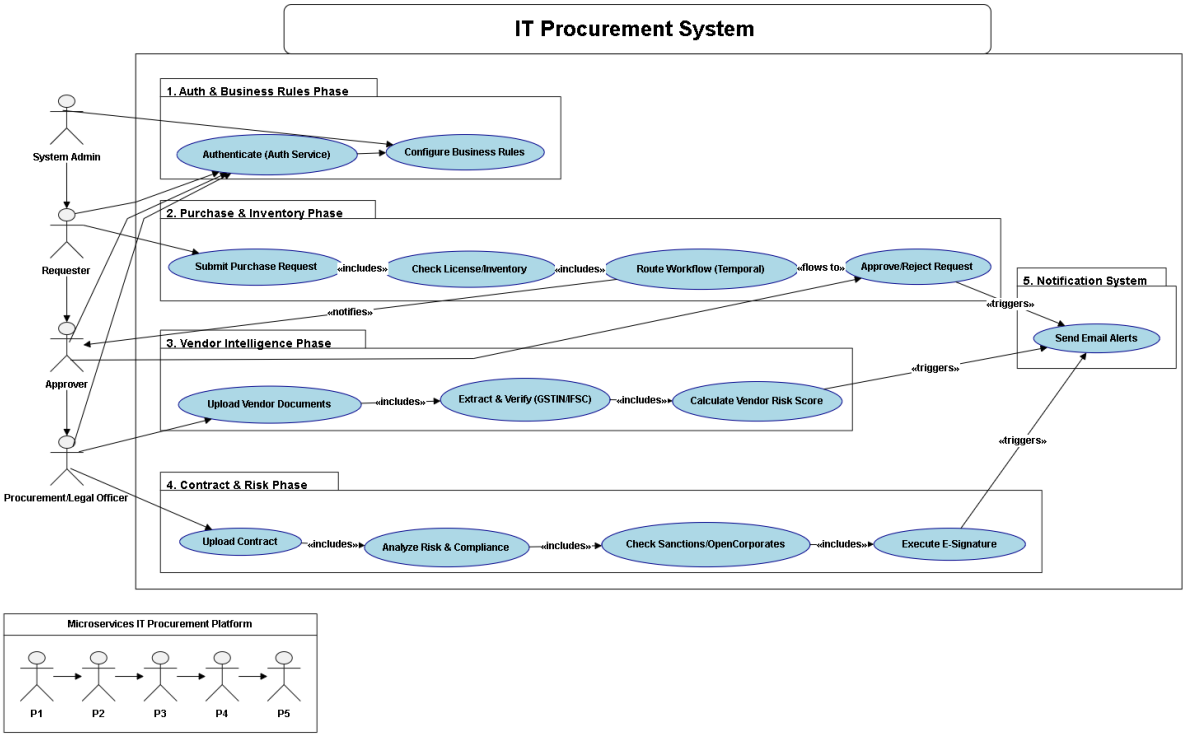


Figure 3.4: Use case diagram of the IT procurement system

1. **Authentication and business rules.** Users sign in through the authentication service, and the spend tiers, SLA and utilisation thresholds that drive routing are held in configuration files.
2. **Purchase and inventory.** A requester submits a purchase request, the system checks and reserves inventory, the approval workflow is routed through Temporal, and the approver approves or rejects.
3. **Vendor intelligence.** Vendor documents are uploaded, extracted and matched, and a vendor risk score is calculated.
4. **Contract and risk.** A contract is generated for the approved request, sent for signature and marked signed when the verified callback arrives, and renewal reminders are scheduled.
5. **Notification.** Approvals, risk scores and signature events trigger email alerts to the people concerned.

3.4.6 Technology Stack

Table 3.4 lists the main technologies. Wherever possible, pretrained open-source components are used, and the only model trained within the project is the vendor risk classifier.

Table 3.4: Technology stack of the proposed system

Layer	**Technology**	**Purpose**
Frontend	React, TypeScript, Vite, Tailwind CSS	Role-based dashboards and System Health page
Gateway	Nginx	Single entry point; routes `/api/<prefix>/` to each service
Backend services	FastAPI (Python), SQLAlchemy, Docker	One containerised service per agent, plus a worker where needed
Messaging	Redpanda (Kafka API), aiokafka	Asynchronous events between agents
Workflow engine	Temporal	Approval workflows, SLA timers, renewal reminders, drift check
Data stores	PostgreSQL, Redis, MinIO	Records, locks and idempotency keys, document files
Document AI	Docling, pdfplumber, Tesseract (pytesseract), rapidfuzz	Layout parsing, OCR, vendor and duplicate matching
Models	scikit-learn (Random Forest), MLflow	Vendor risk scoring and experiment tracking
Security	JWT, bcrypt, HMAC-SHA256, ClamAV	Authentication, password hashing, webhook verification, malware scanning
Monitoring	Prometheus, Grafana	Request rate, latency, error rate and service availability
Notifications	Jinja2 templates, SMTP (Mailpit in development)	Templated email alerts

3.4.7 Key Design Principles

Four principles run through the whole design. The system is *fraud-aware*: uploads are scanned and rejected if the scanner is unavailable, payment-detail changes need a second person's confirmation, and the e-signature webhook is accepted only with a valid HMAC signature and without replay. It is *human-centred*: results below the confidence threshold go to a review queue, a reclaim becomes a request that a person approves, and offboarding flags contracts for review instead of closing them automatically. It is *accountable*: significant actions are written to an audit log, templates fail loudly when a field is missing, and the risk model is monitored for drift without being retrained silently. Finally, it is *observable*: every service exposes Prometheus metrics, a notification log records each email, and a System Health page lets an administrator check the services.

Chapter 4

Design of the System

This chapter describes the requirements, development approach, system models, resource requirements and architecture of the Agentic AI-Based IT Procurement Intelligence Platform. It presents the functional and non-functional requirements, the principal data flows, the system layers and the user-interface design.

4.1 Requirement Engineering

Requirement engineering involves identifying the capabilities expected from the platform, documenting them clearly, and checking that they remain consistent throughout development. In this project it was particularly important because the system is divided into services that must interoperate through shared event formats, database structures, REST response conventions and authentication rules.

4.1.1 Requirement Elicitation

Requirements were identified from the following sources:

- **Review of existing practices and literature:** The review in Chapter 2 and the problem analysis in Chapter 3 identified recurring procurement challenges, including repeated manual data entry, unverified vendor details, duplicate invoices, delayed approvals, underused software licences, missed contract renewals and infrequent vendor-risk reviews.
- **Stakeholder analysis:** The principal users are requesters, approvers and finance users, and system administrators. In the implemented application the four roles are requester, approver, finance and admin.
- **Guide and team discussions:** Discussions with the guides were used to prioritise features and to distinguish the implemented scope from features that depend on external accounts or providers, such as live GSTIN lookups, sanctions screening and a real e-signature provider, which were left as future scope.
- **Review of tools and datasets:** Document-processing tools, workflow engines and relevant datasets were reviewed to understand their expected inputs, outputs and practical limitations. This review led to the decision to drop PaddleOCR in favour of Tesseract after repeated native crashes.

The gathered requirements were then grouped into functional and non-functional requirements.

4.1.2 Software Lifecycle Model

The project followed an iterative and incremental development approach. This approach was suitable because thresholds, approval rules, model behaviour and integration details required testing and refinement during implementation. Each of the four team members owned one service. The main practices were:

- **Contract-first design:** Shared interfaces were agreed before parallel service development. These included the Kafka event envelope and topic payloads, the base database schema, the REST response envelope and the authentication middleware.

- **Service-wise increments:** Work was divided across the document and vendor, approval and inventory, contract and risk, and notification and authentication services.
- **Integration iterations:** The services were connected behind a common API gateway. Integration testing exposed issues in gateway routing, Kafka connectivity, UUID column types and database migrations, which were fixed at the shared level.
- **Continuous testing:** Service-level unit tests and an end-to-end script were used during development rather than postponing all testing until the end, and a later audit pass re-verified each fix against the running stack.
- **Repeatable deployment:** A single `run.sh` script brings up the whole containerised stack from a clean clone, supporting consistent demonstrations.

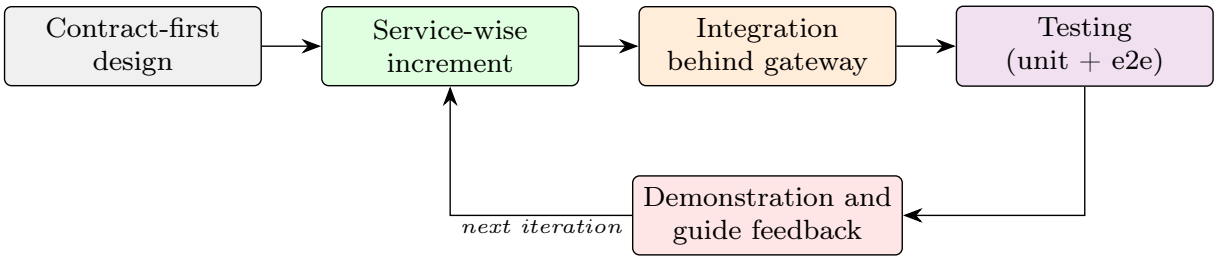


Figure 4.1: Iterative and incremental development cycle

4.1.3 Requirement Analysis

The requirements are grouped into functional requirements, which describe what the system should do, and non-functional requirements, which describe expected qualities and constraints.

4.1.3.1 Functional Requirements

Table 4.1: Functional requirements

ID	Requirement	Service
FR1	Users can log in and receive access and refresh tokens. Protected endpoints verify the token and the user role, except the health and metrics endpoints and the HMAC-authenticated e-signature webhook.	Platform / Authentication
FR2	Uploaded purchase orders, invoices and quotations are scanned for malware before storage. An upload is rejected if the scanner is unavailable.	Document and Vendor
FR3	Documents are classified as purchase orders, invoices or quotations. Vendor, line-item, total, currency, date and document-number fields are extracted.	Document and Vendor

Table 4.1 (continued)

ID	Requirement	Service
FR4	Extracted fields receive confidence scores. Results below the configured threshold (default 0.8) are sent for human review, where fields can be corrected and saved.	Document and Vendor
FR5	Vendor records are matched by normalised name with a fuzzy threshold, and duplicate invoices are detected using vendor, amount tolerance, date window and document-number checks.	Document and Vendor
FR6	Changes to vendor bank or payment details remain pending until verified by a different user with the finance or admin role.	Document and Vendor
FR7	Purchase requests follow approval chains based on spend tiers (auto, manager, manager plus finance), with service-level-agreement (SLA) timers and automatic escalation.	Approval and Inventory
FR8	Approval and rejection decisions are applied through a durable Temporal workflow, and the requester can see the resulting status.	Approval and Inventory
FR9	Inventory is reserved using Redis locks to prevent concurrent requests from reserving the same unit. Oversubscribed requests are split into immediate and back-ordered portions.	Approval and Inventory
FR10	Licence utilisation is calculated from login data, and a reclaim request is created when utilisation falls below the configured threshold.	Approval and Inventory
FR11	Contracts are generated from templates. Renewal type, notice period and end date are extracted, and reminders are scheduled at 60, 30 and 15 days before the notice deadline.	Contract and Risk
FR12	Contracts are sent for signature, and signed-document callbacks are accepted only when the HMAC signature is valid and the callback is not a replay.	Contract and Risk
FR13	Vendor risk is represented by a score and a Low, Medium or High band, with the contributing factors listed and an urgent alert raised for a High band.	Contract and Risk
FR14	The risk model is monitored for drift using the Population Stability Index; automatic retraining is not performed.	Contract and Risk
FR15	Vendor offboarding revokes access, flags active contracts for review, and sets a data-retention flag rather than deleting records.	Contract and Risk
FR16	The notification service consumes events, sends templated email alerts, sends urgent items immediately, batches routine items into digests, and maintains a searchable notification log.	Notification

Table 4.1 (continued)

ID	Requirement	Service
FR17	The frontend provides role-specific navigation and dashboards, a purchase-request wizard, an approval inbox, document review, vendor, contract, risk and inventory views, and notification history.	Frontend
FR18	Significant actions on documents, vendors and contracts are written to an audit log that administrators can query, and an Audit and Activity page and System Health page are provided.	Shared / Frontend

4.1.3.2 Non-Functional Requirements

Table 4.2: Non-functional requirements

Category	Requirement
Security	JWT-based authentication and backend role-based access control; bcrypt password hashing; fail-closed malware scanning; HMAC-verified webhooks; dual control for payment-detail changes; security headers on the gateway.
Reliability	Approval, renewal and drift workflows are durable (Temporal). Kafka consumers are designed for idempotent processing, and webhook replays are rejected using a processed-events table.
Consistency	Endpoints follow a common response envelope, and shared database changes use idempotent service migrations, each with its own migration version table.
Performance	Uploads return promptly while document extraction and other heavy processing run asynchronously in worker containers.
Traceability	Every event carries a unique event ID, source service and timestamp; significant actions are written to an audit log; Prometheus metrics and Grafana dashboards support monitoring of request rate, latency, errors and service availability.
Portability	The stack is containerised and runs on macOS, Linux and Windows through WSL2.
Maintainability	Responsibilities are separated by service, shared contracts are documented, thresholds live in configuration files rather than code, and each service has its own tests.
Explainability	Risk scores display their contributing factors, and low-confidence extraction results are made available for human review.

Constraints and assumptions: Payment execution, ERP integration, GSTIN and IFSC registry verification, sanctions screening, a live e-signature provider and automatic

model retraining are outside the stated scope. The vendor risk training data, the licence usage logs and the sample documents are synthetic.

4.1.4 UML Diagrams and Data Flow Diagrams

4.1.4.1 Use Case Diagram

The use case diagram is presented in Figure 3.4 in Chapter 3. It identifies the Requester, Approver, Finance user and System Admin and groups the main activities into authentication and business rules, purchase and inventory, vendor intelligence, contract and risk, and notifications.

4.1.4.2 Data Flow Diagram – Level 0

The Level 0 context diagram (Figure 4.2) represents the platform as one process and shows its interactions with the external entities: the requester, approver and finance users, the administrator, the vendor, the e-signature provider and the email (SMTP) service.

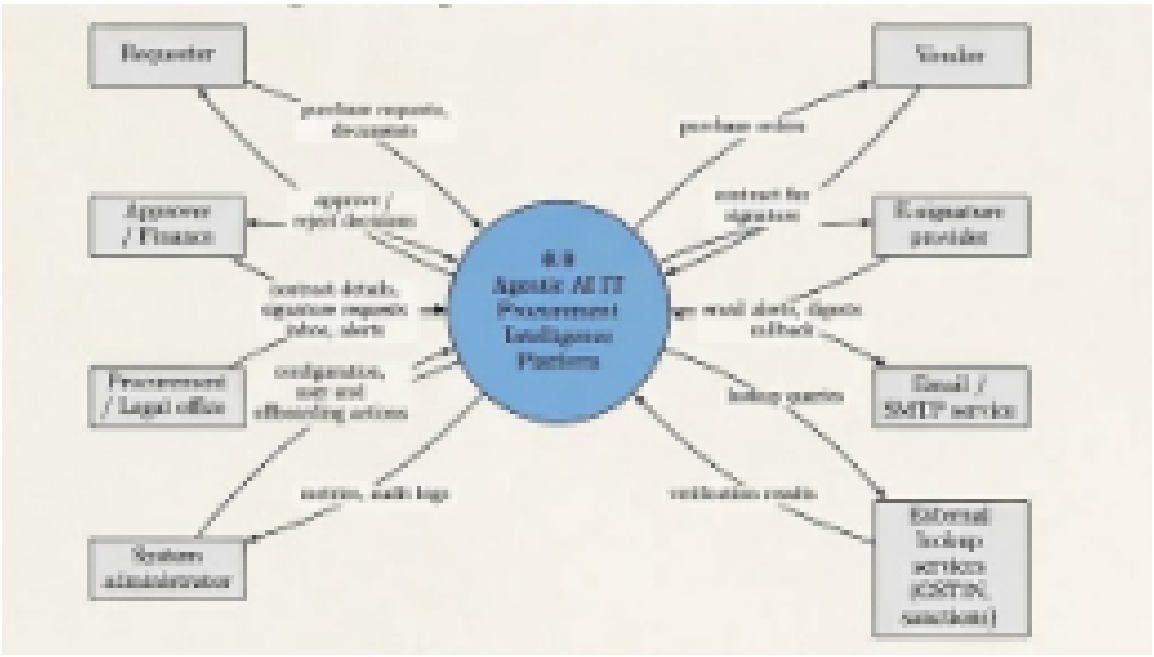


Figure 4.2: Level 0 data flow diagram (context diagram)

4.1.4.3 Data Flow Diagram – Level 1

The Level 1 diagram (Figure 4.3) decomposes the platform into the processes listed in Table 4.3. Processes 2.0 to 5.0 exchange events through the Kafka-compatible messaging layer, and process 6.0 consumes them.

4.1.4.4 Sequence Diagram

The sequence diagram (Figure 4.4) shows the purchase-request journey from the requester and frontend/gateway through document intake, the event bus, the approval API and worker, Temporal, the contract service and the notification service, ending with the signed-contract event and the request-status update.

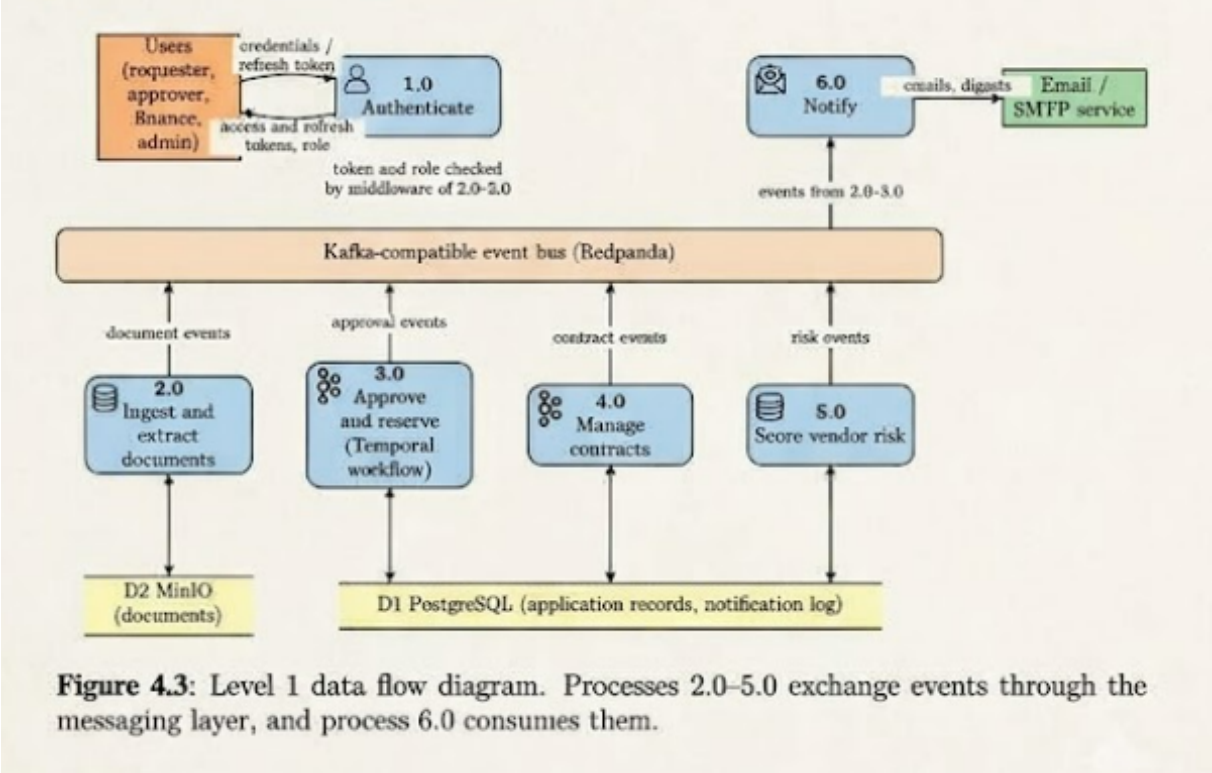


Figure 4.3: Level 1 data flow diagram. Processes 2.0–5.0 exchange events through the messaging layer, and process 6.0 consumes them.

Figure 4.3: Level 1 data flow diagram. Processes 2.0–5.0 exchange events through the messaging layer, and process 6.0 consumes them.

Table 4.3: Level 1 DFD processes

Process	Name	Inputs	Outputs
1.0	Authenticate	Login credentials or refresh token	Access/refresh tokens and user role
2.0	Ingest and extract documents	Uploaded file and request details	Classified/extracted document, vendor match, review-queue items
3.0	Approve and reserve	Purchase request and approval/rejection signal	Decision, stock reservations, escalations, licence reclaim requests
4.0	Manage contracts	Approved request and signed callback	Generated contract, signature status, renewal reminders
5.0	Score vendor risk	Vendor features and outcomes	Risk score, band, contributing factors, drift flag
6.0	Notify	Events from processes 2.0–5.0	Emails, digests, notification log

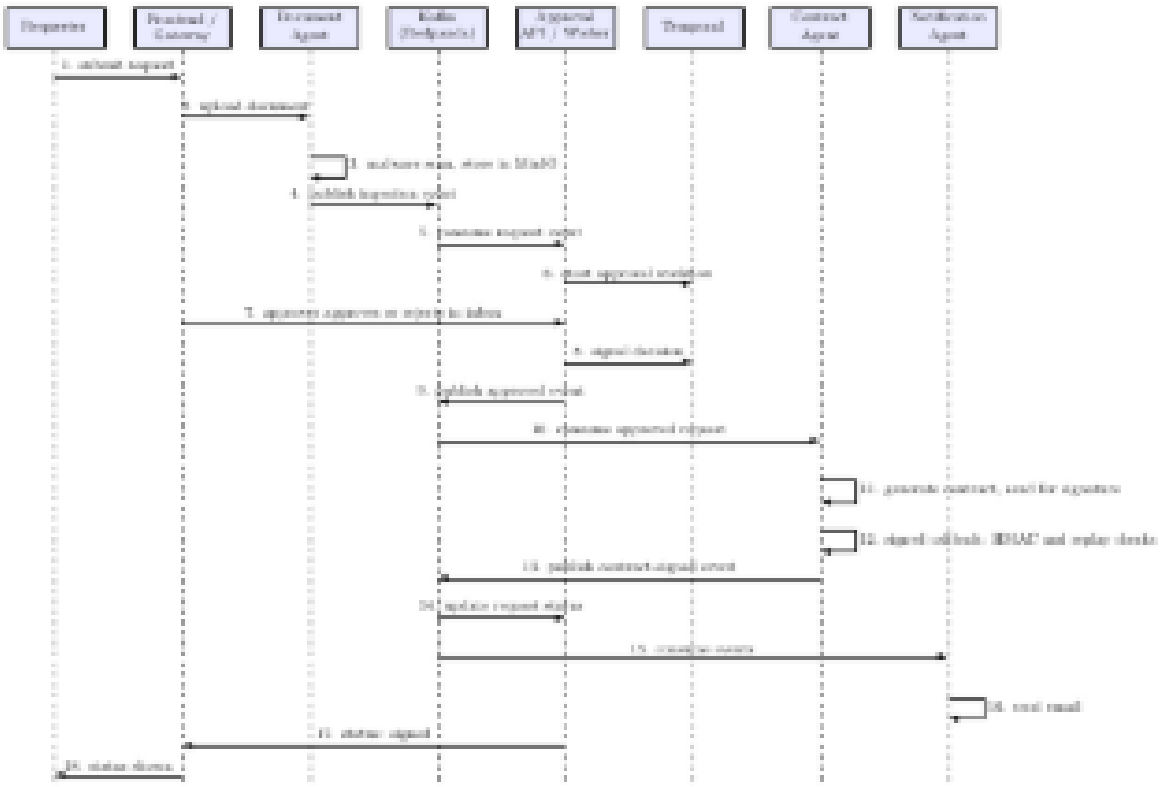


Figure 4.4: Sequence diagram of the purchase-request journey

4.1.4.5 Activity Diagram: Document Processing

The document-processing activity flow (Figure 4.5) begins with upload and malware scanning. The upload is rejected if the file is flagged or the scanner is unavailable. A clean file proceeds through parsing, classification, field extraction, vendor matching, duplicate checking and confidence scoring. Results meeting the threshold are completed; lower-confidence results are sent to a human review queue, and the reviewer's corrections complete the record.

4.1.4.6 Class/Entity Relationship Diagram

The principal entities of the shared schema are vendors, documents, purchase requests, approval history, licences, licence usage, inventory, contracts, risk scores, vendor payment change requests, notification log and the audit log; the authentication service keeps its own user table, and the contract and risk service adds vendor risk features, risk score outcomes and processed webhook events. Figure 4.6 shows the main relationships: a purchase request refers to a vendor and can have approval records and contracts; vendors have documents, contracts, licences, risk scores and payment-change requests; and licences have per-user usage records. The exact table definitions are maintained in the shared database schema and in the service migrations.

4.1.5 Cost Analysis

The project uses open-source software, so no additional software-licence cost is incurred. Development is based on existing student laptops. The principal potential costs are team

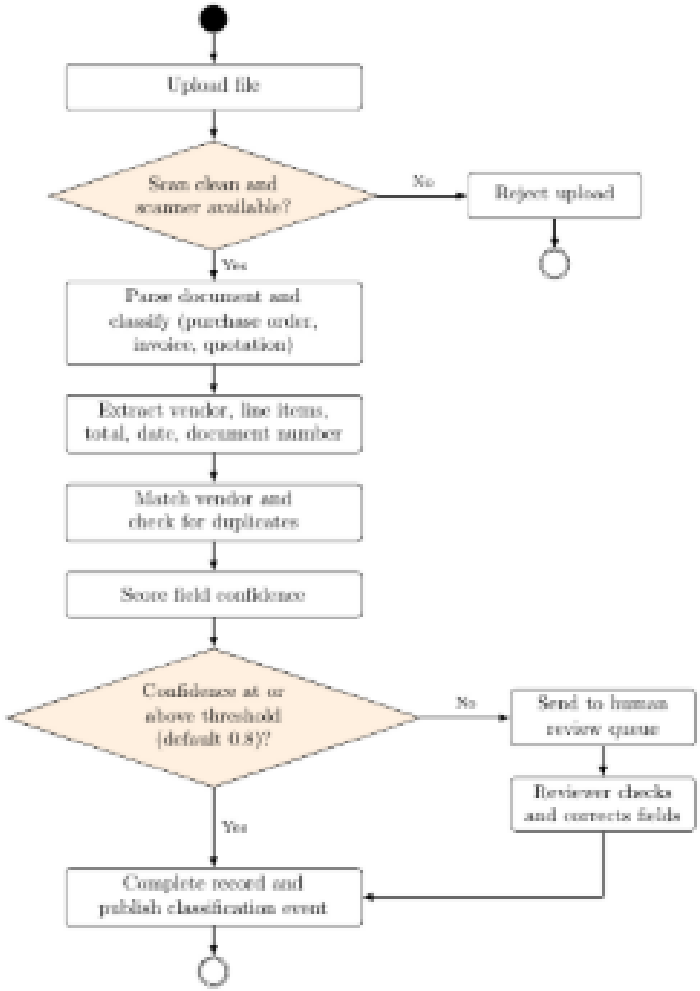


Figure 4.5: Activity diagram of document processing

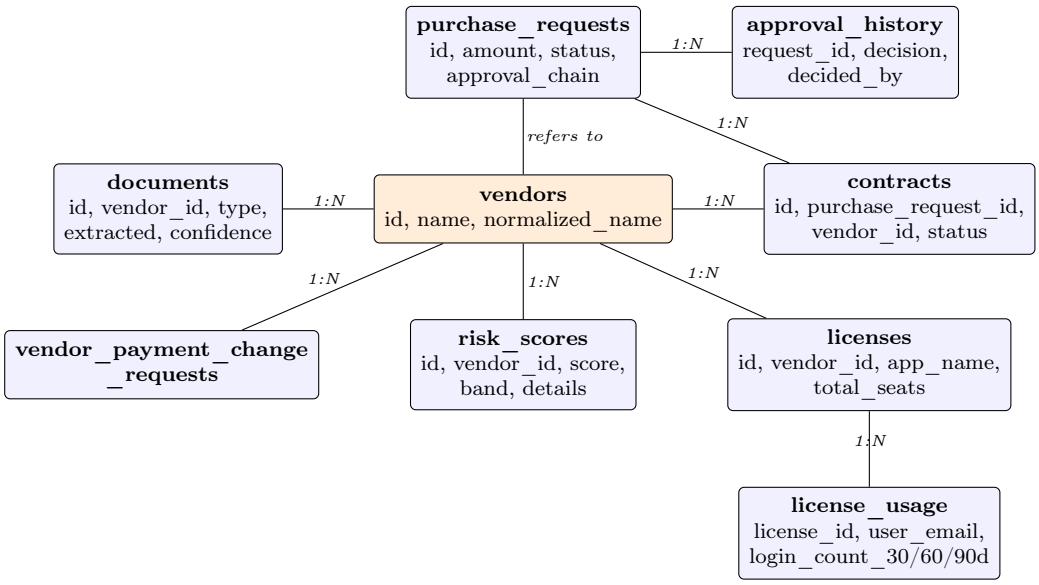


Figure 4.6: Entity relationship diagram (main entities of the shared schema)

effort, external API usage beyond free tiers if live integrations are added later, optional cloud hosting, and a production email service. The estimates in Table 4.4 are indicative and should be updated if actual quotations or usage figures become available.

Table 4.4: Indicative cost analysis

Item	Basis	Indicative Cost
Software licences	Python, FastAPI, PostgreSQL, Redis, Redpanda, MinIO, Temporal, MLflow, Prometheus, Grafana, ClamAV, Docling, Tesseract, React, Docker and other listed open-source tools	Rs. 0
Development hardware	Existing student laptops	Rs. 0 additional
External APIs	None required by the current implementation; future registry or risk-data integrations depend on provider limits	Depends on usage
Team effort	Part-time project work by team members	Not monetised
Optional cloud hosting	Pilot virtual machine with approximately 8 vCPU and 16 GB RAM, plus storage and backups	Approx. Rs. 6,000–10,000 per month
Domain and email delivery	Production SMTP service in place of the development mail tool	A few hundred rupees per month at pilot volume

A meaningful break-even estimate requires measured monthly document volume, average licence costs, and an estimate of time saved or duplicate payments avoided. Those figures are not available, so no return-on-investment claim is made.

4.1.6 Hardware and Software Requirements

The stack runs multiple containers concurrently. The document parser (Docling) and ClamAV can be resource-intensive, so the recommended configuration gives a smoother development and demonstration experience.

Table 4.5: Hardware requirements

Component	Minimum	Recommended
Processor	4 cores	8 cores
Memory	8 GB RAM	16 GB RAM
Storage	25 GB free	50 GB free for images, model files and database volumes
Network	Required for initial builds, image pulls and ClamAV definition updates	Broadband connection
GPU	Not required	Optional for heavier OCR or transformer workloads

Table 4.6: Software requirements

Category	Requirement
Operating system	macOS, Ubuntu/Linux, or Windows 10/11 with WSL2
Container runtime	Docker Engine or Docker Desktop with Docker Compose
Backend	Python with FastAPI, SQLAlchemy, Alembic, pytest, scikit-learn, rapidfuzz and Jinja2
Frontend	React, TypeScript, Vite and Tailwind CSS; the frontend is built in a Node 20 container
Data and messaging	PostgreSQL, Redis, Redpanda (Kafka API) and MinIO
Workflow and ML	Temporal and MLflow
Document processing	Docling, pdfplumber, Tesseract OCR (pytesseract) and ClamAV
Monitoring and mail	Prometheus, Grafana and Mailpit
Gateway	Nginx
Browser	A current version of Chrome, Edge, Firefox or Safari

4.2 System Architecture

The Agentic AI-Based IT Procurement Intelligence Platform follows a modular, event-driven architecture. The system is divided into independent services, each responsible for a specific procurement function. This separation improves maintainability, supports parallel development, and allows background tasks to be processed independently of user-facing requests.

The frontend communicates with backend services through a common API gateway. Synchronous HTTP requests are used for operations that require an immediate response, while inter-service communication primarily uses the Kafka-compatible messaging layer. Temporal supports durable workflows such as approvals, escalations and contract-renewal reminders. PostgreSQL stores application records, Redis supports locking and idempotency keys, and MinIO stores uploaded documents. Background workers process events and execute tasks that may take longer to complete, which allows the application to return responses to users without waiting for every downstream operation to finish.

4.2.1 UI/UX Design

The frontend is implemented as a React and TypeScript single-page application using Vite and Tailwind CSS. Nginx serves the built frontend and routes API requests to the appropriate backend services, so the page and its API calls share one origin and no cross-origin configuration is needed. The interface is designed around the responsibilities of the four implemented roles: requester, approver, finance and administrator. The following design principles guide the user interface:

1. **Role-aware navigation:** Each role sees its own navigation and dashboard. Frontend route checks improve usability but do not replace backend authorisation, which every endpoint enforces independently.

2. **Clear asynchronous status:** Document classification and approval decisions are

applied by background workers, so the interface polls briefly after an action and shows the status of classification, extraction and approvals as they settle.

3. **Transparent system limitations:** Where the backend offers no API for a feature, the interface shows a disabled control or an "unavailable" state instead of pretending the action succeeded.
4. **Human review:** Users can review extracted document fields and their confidence scores and correct them before proceeding.
5. **Demonstration support:** The login screen offers one-click demo-role switching, and a command palette (Ctrl/Cmd+K) searches the requests, vendors, contracts and documents already loaded.

The principal screens and their purposes are summarised in Table 4.7.

Table 4.7: Principal screens and their purposes

Screen	User Role	Purpose
Login	All roles	Sign in or select a demonstration role.
Dashboards	Each role	Role-specific summary: own requests, items awaiting approval, spend overview or system overview.
New Request Wizard	Requester	Five steps: enter details, select a vendor, optionally upload a document with live pipeline status, review extracted fields with confidence, and submit.
Approval Inbox / Queue	Approver, Finance	Review pending requests and approve or reject them according to assigned permissions.
Document Review	Requester, Approver, Admin	Review extracted fields and confidence scores, and correct and save fields.
Vendor Profile	Finance, Admin	View vendor risk score and contributing factors, and the payment-change verification queue.
Contracts	Finance, Admin	Generate contracts, manage the signature workflow, and view renewal timelines.
Risk Dashboard	Finance, Admin	Compare vendor risk bands, recompute scores and check drift.
Inventory	Admin	View hardware stock and software-licence utilisation.
Spend Analysis	Finance	View spending by category.
Audit & Activity, System Health	Admin	Review the audit trail and check the status of the services.
Notification Log	All, subject to permissions	View sent, queued and failed notifications.

4.2.2 Block Diagram

The block diagram (Figure 4.8) represents the major components of the platform and the relationships between them: the frontend, gateway, application services, background

Main-screen wireframes of the platform (login, new-request wizard, approval inbox, vendor profile)

`figures/wireframes.png`

Figure 4.7: Main-screen wireframes of the platform

workers, messaging and workflow components, data stores and monitoring tools. The architectural layers are described in Table 4.8.

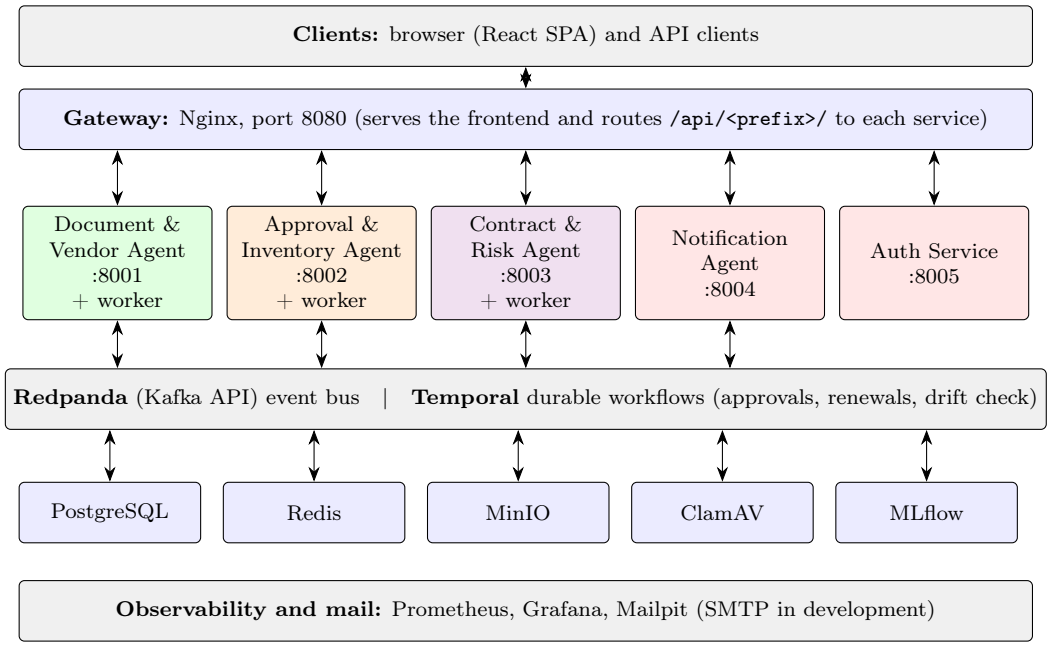


Figure 4.8: Block diagram of the proposed system architecture

4.2.3 Main Data Paths

The principal data paths in the system are described below.

1. **User access:** The browser accesses the frontend through Nginx. API requests are routed to the appropriate backend service after the gateway strips the `/api/<prefix>/` segment.
2. **Document processing:** The Document and Vendor Agent scans an uploaded file for malware. Accepted files are stored in MinIO, and a `document.ingested` event is published. The background worker extracts and classifies the document and publishes the resulting classification and vendor-matching events.
3. **Approval processing:** The Approval and Inventory Agent starts a Temporal workflow for each purchase request. An approve or reject call signals the workflow, and

Table 4.8: Architectural layers

Layer	**Components and Responsibilities**
Clients	The browser-based React application and API clients access the platform through the API gateway on port 8080.
Gateway	Nginx serves the frontend and routes API paths to the appropriate backend services, providing a common entry point.
Application Services	The Document and Vendor Agent runs on port 8001; the Approval and Inventory Agent on port 8002; the Contract and Risk Agent on port 8003; the Notification Agent on port 8004; and the Authentication Service on port 8005. Authentication and role-based middleware protect application endpoints, except for the designated health and metrics endpoints and the HMAC-authenticated e-signature webhook.
Background Workers	The document, approval and contract services each run a worker container that consumes events and runs Temporal workflows and other asynchronous tasks.
Messaging and Orchestration	Redpanda provides a Kafka-compatible event-messaging interface. Temporal supports durable workflows for approvals, escalations and contract renewals.
Data and Platform Services	PostgreSQL stores application records, Redis supports reservation locks and idempotency keys, and MinIO stores documents. ClamAV scans files for malware, MLflow supports model tracking, and Mailpit supports development email testing.
Observability	Prometheus collects metrics and Grafana provides dashboards for monitoring request rates, latency, errors and service availability.

the worker applies the decision, updates the request status and publishes the relevant events.

4. **Contract processing:** The Contract and Risk Agent processes approved requests, generates contracts, and handles signed-document callbacks. The callback is accepted only after the HMAC signature validation and replay checks. A `contract.signed` event is published, allowing the Approval and Inventory Agent to mark the request as fulfilled.
5. **Risk assessment and notifications:** The Contract and Risk Agent publishes risk-score and renewal events. The Notification Agent consumes the applicable events and sends email notifications through the configured SMTP service.
6. **Data access and supporting services:** Application services use PostgreSQL for persistent records. Redis supports inventory reservation locks and idempotency keys, MinIO stores documents, and MLflow tracks the risk-model training runs and drift flags.

These data paths show how synchronous API operations, event-driven communication and durable workflows work together to support procurement activities.

Chapter 5

Result and Discussion

5.1 Screenshots of the System

5.2 Sample Code (of imp part/ main logic)

5.3 Testing

Testing was carried out at three levels: unit tests for each service, an end-to-end integration test through the gateway, and black-box checks of the externally visible behaviour, including the security fixes made during a final audit pass.

5.3.1 Unit Testing

Each service has its own `pytest` suite, run inside that service's Docker image with the source folder mounted (`./scripts/test-service.sh <service>`, or `all`). The suites test pure logic and schemas and need no live infrastructure. In total there are 134 test functions across the five services; the final audit run reported 132 passed and 2 skipped. Table 5.1 summarises what each suite covers.

Table 5.1: Unit tests per service

Service	Tests	Areas covered
document-vendor-agent	42	Document classification, confidence scoring and review gate, Docling extraction, field extraction, vendor matching, agent envelope contracts, ClamAV client, audit router, event schemas, metrics
approval-inventory-agent	30	Spend-tier selection, Redis reservation lock, utilisation and reclaim rules, event schemas, metrics
contract-risk-agent	16	Clause extraction (renewal type, notice period, end date), risk model and bands, event schemas, metrics
notification-agent	38	Digest versus urgent routing, template rendering with strict undefined checking, event schemas, metrics
auth-service	8	Password hashing and verification, JWT creation and validation
Total	**134**	

5.3.2 Integration Testing

The end-to-end test (`make e2e`, script `tests/e2e/run.sh`) exercises the real flow through the running gateway, exactly as a user would, and passes all 12 checks. It logs in as the requester, approver and admin; creates a purchase request; approves it as the approver and polls until the Temporal workflow has applied the decision; confirms that a contract was generated for the approved request; sends the contract for signature; simulates the e-signature provider's callback with a real HMAC signature; confirms the contract is signed; recomputes the vendor's risk score; and confirms that a notification email arrived in Mailpit. This test proves that the services work together and not just individually. Integration of the services also exposed a number of shared-infrastructure problems that were fixed centrally, for example the gateway not stripping the API prefix before forwarding (which

made every route return 404), the event broker advertising an unreachable address, UUID columns typed as strings, and Alembic version tables colliding across services.

5.3.3 Blackbox Testing

Black-box checks test the externally visible behaviour without reference to the code. The cases in Table 5.2 were verified live against the running stack during the audit pass, using `curl` and short Python scripts in addition to the automated suites.

Table 5.2: Black-box test cases

ID	Scenario	Expected result	Outcome
BB1	Full flow: request, approval, contract, signature, risk score, notification	Each step succeeds in order	Pass (12 of 12 checks)
BB2	Requester token calls the payment-change verification endpoint	Request rejected with 403; only finance or admin can verify	Pass
BB3	Requester token reads the approval inbox and the notification log	403 for the requester; 200 for approver and finance	Pass
BB4	Webhook signed with a wrong secret	401 and the contract stays unsigned	Pass
BB5	Ten concurrent identical e-signature webhook deliveries	One 200 signed, nine 200 already_processed, no 500 errors	Pass
BB6	Document review with a corrected vendor name	200 and the vendor is linked (previously a 500 error)	Pass
BB7	Malformed UUID in a path, for example `/vendors/2.1/risk`	Clean 400 error in the standard error envelope	Pass (handler added)

Chapter 6

Conclusion & Future Scope

This report presented the design and implementation of an Agentic AI-Based IT Procurement Intelligence Platform. The study of existing techniques showed that document extraction, clause analysis, approval workflows and risk scoring are usually treated as separate problems. The project joins them into one event-driven system of four cooperating agents: Document and Vendor Intelligence, Approval and Inventory Intelligence, Contract and Risk Analysis, and a Platform agent for notifications and authentication. A document is scanned, parsed, classified, extracted, matched to a vendor and confidence-scored; a request is routed through spend-tier approvals with SLA escalation and race-safe inventory reservation; an approved request becomes a generated contract that is signed through a verified callback and tracked for renewal; and vendors carry an explainable risk score that is monitored for drift. The whole flow runs end to end through one gateway and is supported by 134 unit tests and a 12-step end-to-end test.

The design choices follow a consistent principle. Pretrained or rule-based components are used where they are explainable and sufficient, a trained model is used only for vendor risk, and humans stay in control of low-confidence extractions, payment-detail changes, reclaim decisions and contract reconciliation. Controls such as fail-closed malware scanning, dual control, HMAC-verified and replay-protected webhooks, role-based access and an audit log make the platform suitable for a sensitive financial process.

Limitations. The vendor risk model is trained on a synthetic dataset, the licence usage logs and sample documents are synthetic, and no accuracy claim on real procurement data is made. Document classification and clause extraction are rule-based, so they depend on known phrasings. Vendor identity is checked by fuzzy name matching only. The e-signature step returns a simulated provider reference, because no live provider is connected, although the callback path is real and tested. Payment execution and ERP integration are not included.

Future scope. The following extensions would strengthen the platform:

- Replace Tesseract with PaddleOCR for image OCR on a deployment target where its native engine runs stably, and add LayoutLMv3 as an independent cross-check of extraction.
- Add GSTIN (format, check digit and registry lookup) and IFSC validation as a stronger vendor-identity key than name matching.
- Validate and extend clause extraction against the CUAD benchmark, possibly with a fine-tuned legal language model.
- Feed real external data into the risk model (company registries, financial filings, security certification and TLS grade) and add sanctions screening against a regularly refreshed list.
- Connect a real e-signature provider, and add anomaly-based licence scoring with a grace period before reclaim.
- Add end-to-end request tracing with a correlation identifier, Kafka consumer-lag and approval SLA-breach panels in Grafana, and run the test suites as a gate in the CI pipeline.

Bibliography

[1] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proc. ACM SIGKDD*, 2020.

[2] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proc. ACM Multimedia*, 2022.

[3] C. Auer et al., "Docling technical report," arXiv:2408.09869, 2024.

[4] C. Cui et al., "PaddleOCR 3.0 technical report," arXiv:2507.05595, 2025.

[5] D. Hendrycks, C. Burns, A. Chen, and S. Ball, "CUAD: An expert-annotated NLP dataset for legal contract review," in *NeurIPS Datasets and Benchmarks*, 2021.

[6] I. Chalkidis et al., "LEGAL-BERT: The muppets straight out of law school," in *Findings of EMNLP*, 2020.

[7] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," arXiv:1908.10063, 2019.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE ICDM*, 2008.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017.

[10] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[11] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[13] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007.

Acknowledgement

The success of a project like this, involving high technical expertise, patience, and massive support from guides, is possible when team members work together. We take this opportunity to express our gratitude to those who have been instrumental in the successful completion of this project. We would like to show our appreciation to our project guide **Dr. Jyoti More** and our external guide **Mr. Alok** for their tremendous support and help; without them, this project would not have reached fruition. We would also like to thank our project coordinator **Dr. Rakhi Kalantri** for providing us with regular inputs about documentation and project timeline. A big thanks to our HOD **Dr. M.Kiruthika** for all the encouragement given to our team. We would also like to thank our principal, **Dr. S. M. Khot**, and our college, **Fr. C. Rodrigues Institute of Technology, Vashi**, for giving us the opportunity and the environment to learn and grow.

Project Group Members:

1. More Anjali Dinesh, 1023207

2. Sankaye Vaidehi Jagdeep, 1023239

3. Sawant Niraj Ravindra, 1023240

4. Shete Anooj, 1023250

Appendix A : Timeline Chart

This is appendix A

Appendix B

The project team has identified the following Sustainable Development Goal(s) (SDG) relevant to the proposed project. The mapping should demonstrate a clear and meaningful relationship between the project problem, proposed solution, and the selected SDG(s).

SDG Mapping of the Project

Sr. No.	**SDG No.**	**Complete Name of SDG**	**Rationale / Contribution of the Project**
1	9	Industry, Innovation and Infrastructure	The platform applies AI, event-driven software architecture and workflow automation to a core enterprise process, improving the technological capability and efficiency of IT procurement.
2	12	Responsible Consumption and Production	Duplicate-invoice detection, inventory reservation and the reclaim of underused software licences reduce wasted purchases and unused resources.
3	16	Peace, Justice and Strong Institutions	Role-based access, dual control of payment-detail changes, audit logging and explainable decisions support accountable, transparent and fraud-resistant institutions.

Guidelines for Students:

- Identify only the SDG(s) that have a direct and meaningful connection with the project.
- Mention the official SDG number and complete name.
- Provide a clear rationale explaining how the project contributes to the selected SDG.
- Avoid mapping an SDG merely because the project falls within a particular technology domain.
- The SDG mapping should be discussed and validated with the Project Guide.

Signature of Project Guide Name of Project Guide: Dr. Jyoti More

Date:

Appendix C

Appendix C: Complex Engineering Problem Identification and Evaluation

COMPLEX ENGINEERING PROBLEM IDENTIFICATION AND EVALUATION

The Washington Accord emphasizes the ability of engineering graduates to identify, analyse and solve complex engineering problems using appropriate engineering knowledge, modern tools, multidisciplinary thinking and analytical reasoning. The Major Project provides an opportunity for students to demonstrate these abilities through a structured and evidence-based project.

This appendix provides a structured mechanism to assess the complexity of the proposed Major Project. The assessment is initially carried out by the student team and subsequently verified by the Project Guide.

A. Project Details

Project Title	Agentic AI Procurement Solution
Group / Team No.	
Name of Student(s)	More Anjali Dinesh (1023207), Sankaye Vaidehi Jagdeep (1023239), Sawant Niraj Ravindra (1023240), Shete Anooj (1023250)
Project Guide	Dr. Jyoti More
Academic Year	

B. Complexity Evaluation

For each parameter, the student team shall assign a score from 0 to 3 and provide a suitable justification or evidence. The Project Guide shall review the student's assessment and record the verified score in the final column.

Sr. No.	Complexity Parameter	Student Score	Student Justification / Evidence	Guide Score
1	Problem novelty and research depth	2	Integrates document AI, workflows and risk scoring in one governed pipeline; individual techniques are established.	
2	Multidisciplinary knowledge requirement	3	Combines document processing, machine learning, distributed systems, workflow engines, security and web development.	
3	System design complexity	3	Five services plus workers, shared event and database contracts, durable workflows and a gateway.	
4	Technology integration	3	FastAPI, Redpanda, Temporal, PostgreSQL, Redis, MinIO, ClamAV, MLflow, Prometheus, Grafana, React in one containerised stack.	
5	Data analysis and experimentation	2	Risk model trained and tracked in MLflow with drift monitoring; data is synthetic, and extraction was tested on 21 sample documents.	
6	Societal / industrial relevance	2	Addresses duplicate payments, unused licences and fraud risks that affect organisations.	
7	Stakeholder involvement	2	Four user roles modelled; guidance from an external industry guide.	
8	Sustainability considerations	1	Reduces wasted purchases and unused licences.	
	Total Score	**18 / 24**		**/ 24**

Scoring Guideline: 0 – Not applicable / negligible; 1 – Basic level of complexity; 2 – Moderate level of complexity; 3 – Significant / high level of complexity. Maximum Score: 24.

C. Final Project Classification

The final classification shall be based on the score verified by the Project Guide.

Score Range	Project Classification
0–8	Simple Engineering Problem
9–16	Moderately Complex Engineering Problem
17–24	Complex Engineering Problem

Final Classification:

☐ Simple Engineering Problem ☐ Moderately Complex Engineering Problem ☐ Com-

plex Engineering Problem

Project Guide Signature Date:

Appendix D: Project Outcomes

Add publication details, project competition participation/winning